Iteración 5 — Tests de integración y fix de potencia

EcoWarm · Grupo 4 · Tareas de la Persona 2 · rama `implementacion4`, commits `c454323` y `58ebc3b` · 2026-10-26

Qué se hizo

- **Base de tests de integración**: el engine completo (Spring, listener MQTT, core, cliente REST de switches) contra un **broker MQTT embebido** (Moquette) y **switches HTTP falsos** que implementan el estándar v2. No hace falta Docker.
- **24 tests de integración** en tres clases: control según la carga, situaciones iniciales y arranque / parada, y cambios de configuración en ejecución.
- **Fix en el engine**: los tests encontraron que, al redistribuir la potencia, el engine podía encender una habitación antes de apagar otra y superar por un instante la potencia contratada. Ahora manda primero los OFF.

Archivos

Archivo	Tipo	Líneas	Para qué
`modules/engine/pom.xml`	modificado	+15	Agrega Moquette solo para tests.
`.../adaptador/CoreAdapter.java`	modificado	+10	Fix: OFF antes que ON.
`.../adaptador/CoreAdapterTest.java`	modificado	1	Actualiza el orden esperado de los comandos.
`.../integracion/BrokerMqttEmbebido.java`	nuevo	164	Broker MQTT en el proceso del test, que se puede bajar y levantar.
`.../integracion/SwitchesHttpFalsos.java`	nuevo	134	Servidor HTTP de switches (§3/§4), con estado, caídas e historial.
`.../integracion/IntegracionBase.java`	nuevo	297	Clase base: arma el engine y trae los helpers.
`.../integracion/ControlCargaTest.java`	nuevo	156	6 tests: carga no limitante y limitante.
`.../integracion/ArranqueParadaTest.java`	nuevo	184	11 tests: situaciones iniciales, INICIAR y DETENER.
`.../integracion/CambioConfiguracionTest.java`	nuevo	174	7 tests: cambios de configuración en caliente.

El prefijo `...` es `modules/engine/src/main/java/uy/edu/utec/iiss/engine` para `CoreAdapter.java` y `modules/engine/src/test/java/uy/edu/utec/iiss/engine` para el resto.

Cómo funciona un test de integración

1. El broker embebido y los switches falsos se levantan una vez para toda la corrida.
2. Cada test arranca un engine nuevo, que se conecta al broker y guarda el inventario en un archivo temporal.
3. El test carga el sitio con `PUT /sitio` e inicia el control con `POST /control/comandos`, como lo haría el simulador.

4. Las lecturas se publican por MQTT con el formato del termostato (§2). El helper espera a que el engine esté suscrito antes de publicar y a que el core registre la lectura.
5. Se verifica lo que recibieron los switches por HTTP (§3/§4) y el motivo de la decisión del core.

Los 24 tests

Clase	Test	Qué verifica
`ControlCargaTest`	cargaNoLimitante	Con potencia de sobra se encienden todas las que tienen frío y cada una se apaga al llegar a su temperatura.
	habitacionCalienteNoSeEnciende	Una habitación por encima de su esperada nunca recibe ON.
	cargaLimitante	3 × 1.5 kW con 3.7 kW contratados: solo entran las dos de mayor déficit; la tercera queda SIN_POTENCIA_DISPONIBLE.
	cargaLimitante_redistribuye	La potencia que libera una habitación satisfecha pasa a la que esperaba, sin superar nunca la contratada (el test que encontró el bug).
	habitacionQueExcedeLaContratada	Una habitación de 4 kW con 3.7 kW contratados nunca se enciende (EXCEDE_CONTRATADA).
	franjaPunta	En la franja punta no se calefacciona aunque sobre potencia (TARIFA_PUNTA).
`ArranqueParadaTest`	sinIniciarNoSeComanda	Sin INICIAR no hay comandos ni consultas a los switches.
	iniciarConsultaLosSwitches	Al INICIAR se consulta el estado real de cada switch (§4).
	encendidaSinLecturas_seApaga	Calefacción encendida de antes y sin lecturas: se apaga (SIN_LECTURA).
	encendidasDeAntes_seAjustanConLaPrimeraLectura	Con la primera lectura queda encendida la fría y se apaga la caliente.
	lecturasAnterioresAIniciar_noCuentan	Una lectura que llegó con el control detenido no se usa.
	iniciarSinSitio	INICIAR sin sitio no falla, y el sitio que llega después se controla.
	iniciarEnFranjaPunta	Iniciar en punta apaga lo que estaba encendido.
	detenerDejaDeComandar	DETENER corta los comandos (incluida la reconciliación) y deja los switches como estaban.
	detenerIdempotente	DETENER repetido, o sin haber iniciado, no da error.
	reiniciarRetomaElControl	Al reiniciar se corrige lo que cambió mientras estaba detenido.
	cambioManualSeCorrige	Con el control en marcha, un cambio manual se corrige en la reconciliación.
`CambioConfiguracionTest`	bajarTemperaturaEsperada_apaga	Bajar la esperada por debajo de la actual apaga la habitación.
	subirTemperaturaEsperada_enciende	Subir la esperada enciende la habitación con la lectura que ya tenía.
	agregarHabitacion	Una habitación nueva se suscribe a su termostato y se controla.
	eliminarHabitacion	Una habitación borrada deja de escucharse y de comandarse.

	bajarPotenciaContratada	Con menos potencia contratada se apaga la de menor déficit.
	cambiarSwitch	Cambiar el switch de una habitación: se comanda el nuevo y el viejo no se toca más.
	cambiarTopic	Cambiar el topic del termostato: se escucha el nuevo y se deja el viejo.

El bug que encontraron los tests

> **Síntoma.** En `cargaLimitante_redistribuye` hay tres habitaciones de 1.5 kW con 3.7 kW contratados; living y dormitorio están encendidas y el estudio espera. Cuando el living llega a su temperatura, el core decide en el mismo momento "living OFF, estudio ON". Como el estudio iba primero en el inventario, el engine mandaba su ON antes que el OFF del living. Los switches recibieron:
>
> `sw-living ON → sw-dormitorio ON → sw-estudio ON → sw-living OFF`
>
> Entre el tercer y el cuarto comando estuvieron encendidas las tres: **4.5 kW con 3.7 kW contratados**. En una casa real eso puede hacer saltar la llave general. El estado final era correcto, por eso ningún test anterior lo había detectado.

> **Corrección.** `CoreAdapter.enviarPendientes()` ahora recorre las habitaciones dos veces: primero manda todos los OFF y después todos los ON. Así, la potencia que se libera ya está apagada cuando se enciende la siguiente. El test verifica el pico reconstruyendo el orden real de los comandos (`picoDePotencia`).

Cómo se corren

- `mvn verify`: corre todos los tests (unitarios y de integración) y verifica el formato.
- `mvn test -pl modules/engine -Dtest="ControlCargaTest,ArranqueParadaTest,CambioConfiguracionTest"`: solo los de integración.
- `./scripts/test.sh engine`: lo mismo, dentro de Docker.

Resultado de la última corrida (dos veces seguidas): **BUILD SUCCESS**, 116 tests del core y 80 del engine (antes 56), 0 fallas.

Para la Persona 3 (fallas y robustez)

- Caída y recuperación del broker: `BROKER.detener()` y después `BROKER.iniciar()`. El broker vuelve en el mismo puerto.
- Switch que no responde o falla REST: `SWITCHES.caido("sw-living", true)`, que responde 503 a todo; con `false` vuelve a responder.
- Termostato que deja de funcionar: dejar de llamar a `temperatura(...)` y bajar `control.vigencia-lectura-segundos` en las propiedades del test (la vigencia se mide en tiempo real: un `factorTiempo` alto no la acorta).
- Pérdida de mensajes: publicar con `BROKER.publicar(topic, payload)` payloads inválidos, o publicar con el broker caído.

Cómo leer el detalle

En las páginas siguientes cada archivo aparece completo, con su número de línea real. A la izquierda está el código y a la derecha la explicación de esa línea o del grupo de líneas que forman una misma instrucción. En los archivos modificados se muestra solo la zona cambiada, con un poco de contexto, y las líneas agregadas o cambiadas aparecen en verde.

1. Dependencia de test: broker MQTT embebido

`modules/engine/pom.xml` — líneas 55–76 (en verde, las líneas agregadas o cambiadas)

```
            <artifactId>junit-platform-launcher</artifactId>
            <scope>test</scope>
        </dependency>
```

Contexto: la dependencia que ya existía (`junit-platform-launcher`). Lo nuevo empieza abajo.

```
        <!-- Broker MQTT embebido para los tests de integración (src/test/.../integracion):
             se puede levantar y tirar desde el test. reload4j se excluye porque el
             logging lo da Logback (Spring Boot). -->
```

Comentario XML que explica para qué se agrega Moquette y por qué se excluye reload4j.

```
        <dependency>
```

Abre la declaración de la nueva dependencia.

```
            <groupId>io.moquette</groupId>
            <artifactId>moquette-broker</artifactId>
```

Coordenadas Maven: `io.moquette:moquette-broker` , un broker MQTT escrito en Java que puede correr dentro del mismo proceso del test, sin Docker ni Mosquitto instalado.

```
            <version>0.17</version>
```

Versión fija `0.17` (la última publicada en Maven Central), para que el build sea reproducible.

```
            <scope>test</scope>
```

`scope test` : solo existe en el classpath de los tests. No entra al jar del engine ni a la imagen Docker.

```
            <exclusions>
                <exclusion>
                    <groupId>org.slf4j</groupId>
                    <artifactId>slf4j-reload4j</artifactId>
                </exclusion>
            </exclusions>
```

Se excluye `slf4j-reload4j` . Moquette trae su propio "binding" de logging; Spring Boot ya usa Logback, y tener dos bindings provoca advertencias y logs duplicados.

```
        </dependency>
```

Cierra la dependencia.

```
        <dependency>
            <groupId>uy.edu.utec.iiss</groupId>
            <artifactId>core</artifactId>
            <version>1.0.0</version>
```

Contexto: la dependencia al módulo `core` , que ya estaba.

2. Fix: mandar los OFF antes que los ON

`modules/engine/src/main/java/uy/edu/utec/iiss/engine/adaptador/CoreAdapter.java — líneas 310–366 (en verde, las líneas agregadas o cambiadas)`

310 `    }` 311	Contexto: fin del método anterior (`consultar`).
312 `    /**` 313 `     * Primero los OFF y después los ON: si una habitación cede su potencia a otra, apagar antes de` 314 `     * encender evita que por un instante se supere la potencia contratada.` 315 `     */`	Javadoc nuevo. Explica la regla: en una misma pasada primero se apaga y después se enciende. Si una habitación le cede su potencia a otra, encender antes de apagar dejaría por un instante más carga que la contratada (en el test, 4.5 kW con 3.7 kW contratados).
316 `    private void enviarPendientes() {`	`enviarPendientes()` sigue siendo el método que llama la pasada; su firma no cambió, así que el resto del adaptador no se toca.
317 `        List<HabitacionControlada> candidatas;` 318 `        synchronized (this) {` 319 `            candidatas = List.copyOf(habitaciones.values());` 320 `        }`	Igual que antes: se toma una copia de las habitaciones bajo el candado, para no recorrer el mapa mientras otro hilo lo modifica.
321 `        enviarPendientes(candidatas, Accion.OFF);`	NUEVO. Primera vuelta: se mandan solo los comandos OFF pendientes.
322 `        enviarPendientes(candidatas, Accion.ON);`	NUEVO. Segunda vuelta: recién ahora se mandan los ON. Para ese momento la potencia liberada ya está apagada en el switch.
323 `    }`	NUEVO. Cierra el método público.
324	Línea en blanco de separación.
325 `    private void enviarPendientes(List<HabitacionControlada> candidatas, Accion soloEsta) {`	NUEVO. Lo que antes era el cuerpo de `enviarPendientes()` pasa a ser un método privado con un parámetro `soloEsta` : la acción (OFF u ON) que se manda en esta vuelta.
326 `        for (HabitacionControlada h : candidatas) {` 327 `            Accion accion;` 328 `            synchronized (this) { // la decisión vigente se lee justo antes de mandarla` 329 `                if (!habilitado) {` 330 `                    return;` 331 `                }`	Igual que antes: recorre las candidatas; bajo el candado verifica que el control siga habilitado (si se DETUVO a mitad de la pasada, corta).
332 `                Decision d = deseado.get(h.id());`	Igual que antes: lee la decisión vigente de la habitación justo antes de mandarla.
333 `                if (!vigente(h)` 334 `                        || d == null`	Igual que antes: descarta si la habitación ya no está configurada con ese switch o si no hay decisión.
335 `                        || d.accion() != soloEsta`	NUEVO. Descarta las decisiones que no corresponden a esta vuelta (en la vuelta de OFF se saltean los ON y viceversa).
336 `                        || coincide(estadoConocido.get(h.id()), d.accion())` 337 `                        || enEspera(h.id())) {` 338 `                    continue;` 339 `                }`	Igual que antes: descarta si el switch ya está en el estado deseado o si está en espera por fallas recientes.

```
            accion = d.accion();
        }

        try {
            EstadoSwitch resultado = switches.comandar(h.urlSwitch(), h.idSwitch(), accion);
            synchronized (this) {
                if (!vigente(h)) {
                    continue;
                }
                estadoConocido.put(h.id(), resultado);
                if (noReintentarAntesDe.remove(h.id()) != null) {
                    log.info(
                            "Switch '{}' (habitación '{}') volvió a responder.",
                            h.idSwitch(),
                            h.id());
                }
            }
            log.info("Switch '{}' (habitación '{}') -> {}", h.idSwitch(), h.id(), accion);
        } catch (RuntimeException e) {
            synchronized (this) {
                if (vigente(h)) {
                    estadoConocido.remove(h.id());
                    registrarFalla(h, "enviar " + accion + " a", e);
                }
            }
        }
    }
}
```

Igual que antes: guarda la acción a mandar y suelta el candado (la red nunca se usa con el candado tomado).

Igual que antes: manda el comando por REST; si responde, actualiza el estado conocido y registra si "volvió a responder"; si falla, olvida el estado y programa el reintento.

Igual que antes: cierre del bucle y del método, y comienzo de `vigente()`.

3. Ajuste de un test existente

`modules/engine/src/test/java/uy/edu/utec/iiss/engine/adaptador/CoreAdapterTest.java — líneas 352–360 (en verde, las líneas agregadas o cambiadas)`

```
352
353             liberar.countDown();
354             long limite = System.nanoTime() + Duration.ofSeconds(2).toNanos();
355             while (comandos.size() < 2 && System.nanoTime() < limite) {
356                 Thread.sleep(10);
357             }
```

Contexto: el test `hiloDeSwitches_unSwitchLentoNoDemoraAlQueLlama` libera el switch lento y espera a que salgan los dos comandos.

```
358             assertEquals(List.of("sw-dorm:OFF", "sw-living:ON"), comandos); // OFF antes que ON
```

CAMBIO. Antes esperaba `[sw-living:ON, sw-dorm:OFF]`; con el fix, el OFF sale primero, así que ahora espera `[sw-dorm:OFF, sw-living:ON]`. El test verificaba que el switch lento no bloquea al que llama; el orden estaba fijado por casualidad.

```
359         }
360     }
```

Cierre del bloque y del test.

4. Broker MQTT embebido

modules/engine/src/test/java/uy/edu/utec/iiss/engine/integracion/BrokerMqttEmbebido.java — líneas 1–164

```java
package uy.edu.utec.iiss.engine.integracion;

```

Paquete nuevo `integracion`, dentro de los tests del engine (la consigna pide los tests de integración junto al código).

```java
import io.moquette.broker.Server;
import io.moquette.broker.config.MemoryConfig;
import io.moquette.interception.AbstractInterceptHandler;
import io.moquette.interception.messages.InterceptConnectionLostMessage;
import io.moquette.interception.messages.InterceptDisconnectMessage;
import io.moquette.interception.messages.InterceptSubscribeMessage;
import io.moquette.interception.messages.InterceptUnsubscribeMessage;
```

Imports de Moquette: el servidor (`Server`), su configuración en memoria (`MemoryConfig`) y las clases para "interceptar" eventos del broker (conexiones, suscripciones, desuscripciones).

```java
import java.io.IOException;
import java.io.UncheckedIOException;
import java.net.ServerSocket;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.Map;
import java.util.Properties;
import java.util.Set;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;
```

Imports del JDK: manejo de errores de E/S, `ServerSocket` para buscar un puerto libre, charset UTF-8 y colecciones concurrentes.

```java
import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;

```

Imports de Paho, el mismo cliente MQTT que usa el engine. Acá se usa para publicar como si fuera un termostato.

```java
/**
 * Broker MQTT en el mismo proceso (Moquette), en un puerto fijo durante toda la corrida: se puede
 * {@link #detener()} y volver a {@link #iniciar()} para simular que el broker cae y se recupera.
 *
 * <p>Sabe qué cliente está suscrito a qué topic, así el test espera a que el engine se suscriba
 * antes de publicar (un mensaje publicado antes se pierde: la sesión es limpia).
 */
```

Javadoc de la clase: broker en el mismo proceso, en un puerto fijo durante toda la corrida, que se puede detener y volver a iniciar (para simular caídas). Además sabe quién está suscrito a qué topic.

```java
public final class BrokerMqttEmbebido {

```

Clase `final` : es una utilidad de test, no se hereda.

```java
    private final int puerto = puertoLibre();
```

El puerto se elige una sola vez al crear el objeto. Si el broker se reinicia, vuelve en el mismo puerto y el engine puede reconectarse a la misma URL.

```java
    private final Map<String, Set<String>> topicsPorCliente = new ConcurrentHashMap<>();
```

Mapa clienteMQTT → topics a los que está suscrito. Es concurrente porque lo escriben los hilos de Moquette y lo lee el test.

```java
    private Server server;
    private MqttClient publicador;

```

El servidor Moquette activo (null = caído) y el cliente Paho que publica las lecturas.

```
39      public String url() {
40          return "tcp://127.0.0.1:" + puerto;
41      }
42
43      public synchronized void iniciar() {
44          if (server != null) {
45              return;
46          }
47          Properties p = new Properties();
48          p.setProperty("host", "127.0.0.1");
49          p.setProperty("port", String.valueOf(puerto));
50          p.setProperty("websocket_port", "disabled");
51          p.setProperty("allow_anonymous", "true");
52          p.setProperty("persistence_enabled", "false");
53          p.setProperty("telemetry_enabled", "false");
54          Server nuevo = new Server();
55          try {
56              nuevo.startServer(new MemoryConfig(p), List.of(new Espia()));
57          } catch (IOException e) {
58              throw new UncheckedIOException(e);
59          }
60          server = nuevo;
61      }
62
63      public synchronized void detener() {
64          cerrarPublicador();
65          if (server != null) {
66              server.stopServer();
67              server = null;
68          }
69          topicsPorCliente.clear();
70      }
71
72      public boolean haySuscriptor(String topic) {
73          return topicsPorCliente.values().stream().anyMatch(t -> t.contains(topic));
74      }
75
76      /** Publica una lectura con el formato del termostato (§2), QoS 1, sin retener. */
77      public synchronized void publicarTemperatura(String topic, double temperaturaC) {
78          String json =
79                  "{\"id\":0,\"tC\":%s,\"tF\":%s,\"ts\":%s}"
```

URL del broker con el formato que espera el engine en `mqtt.broker.url`.

`iniciar()` es idempotente: si ya está levantado no hace nada. `synchronized` evita que dos hilos lo levanten a la vez.

Configuración del broker como `Properties`.

Escucha solo en 127.0.0.1 (no expone nada a la red) y en el puerto elegido.

Desactiva el puerto WebSocket, que no se usa.

Acepta clientes sin usuario ni contraseña, igual que el broker del entorno.

Sin persistencia en disco: cada corrida arranca limpia y no deja archivos.

Desactiva la telemetría de Moquette (no manda datos a internet).

Crea el servidor y lo arranca con la configuración y con el interceptor `Espia` (definido abajo).

Si no puede levantar (puerto ocupado, por ejemplo), el error se propaga sin checked exception.

Recién cuando arrancó bien se guarda como servidor activo.

`detener()`: primero cierra el cliente publicador para no dejar una conexión colgada.

Si había servidor, lo baja. Desde ahí, para el engine el broker "se cayó".

Se olvidan todas las suscripciones: con el broker abajo no hay ninguna viva.

`haySuscriptor(topic)`: true si algún cliente conectado (en la práctica, el engine) está suscrito a ese topic. Los tests lo usan para no publicar antes de tiempo.

Publica una lectura con el formato del termostato del estándar (§2), con QoS 1 y sin retener.

Arma el JSON `{"id":0,"tC":…,"tF":…,"ts":…}` con los mismos campos que manda el termostato simulado.

```
                    .formatted(
                            temperaturaC,
                            temperaturaC * 9 / 5 + 32,
                            System.currentTimeMillis() / 1000.0);
        publicar(topic, json);
    }

    public synchronized void publicar(String topic, String payload) {
        try {
            if (publicador == null || !publicador.isConnected()) {
                cerrarPublicador();
                publicador =
                        new MqttClient(
                                url(),
                                "termostato-test-" + UUID.randomUUID(),
                                new MemoryPersistence());
                MqttConnectOptions opciones = new MqttConnectOptions();
                opciones.setCleanSession(true);
                publicador.connect(opciones);
            }
            publicador.publish(topic, payload.getBytes(StandardCharsets.UTF_8), 1, false);
        } catch (MqttException e) {
            throw new IllegalStateException("No se pudo publicar en " + topic, e);
        }
    }

    private void cerrarPublicador() {
        if (publicador == null) {
            return;
        }
        try {
            if (publicador.isConnected()) {
                publicador.disconnect();
            }
            publicador.close();
        } catch (MqttException e) {
            // el broker ya no está: no hay nada que cerrar
        }
        publicador = null;
    }
```

Rellena los valores: la temperatura en °C, su equivalente en °F y el instante actual en segundos (el engine lo parsea pero decide con su propio reloj).

Delega en `publicar`.

`publicar(topic, payload)`: si no hay cliente o se desconectó (por ejemplo tras reiniciar el broker), crea uno nuevo.

Cierra el viejo y crea un cliente Paho con un id único por conexión y persistencia en memoria.

Sesión limpia, conexión al broker y cierre del `if`.

Publica los bytes UTF-8 con QoS 1 (entrega al menos una vez) y `retained=false`, para que una lectura de un test no le llegue al siguiente.

Si el broker no está, el error se informa con el topic en el mensaje.

`cerrarPublicador()`: si no hay cliente, no hay nada que hacer.

Si sigue conectado, se desconecta; después se liberan sus recursos.

Si el broker ya estaba caído, la desconexión falla; se ignora a propósito.

Se olvida el cliente; el próximo `publicar` crea otro.

```
    private static int puertoLibre() {
        try (ServerSocket s = new ServerSocket(0)) {
            return s.getLocalPort();
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }

    /** Anota las suscripciones vivas de cada cliente. */
    private final class Espia extends AbstractInterceptHandler {

        @Override
        public String getID() {
            return "espia-suscripciones";
        }

        @Override
        public void onSessionLoopError(Throwable error) {}

        @Override
        public void onSubscribe(InterceptSubscribeMessage m) {
            topicsPorCliente
                    .computeIfAbsent(m.getClientID(), c -> ConcurrentHashMap.newKeySet())
                    .add(m.getTopicFilter());
        }

        @Override
        public void onUnsubscribe(InterceptUnsubscribeMessage m) {
            Set<String> topics = topicsPorCliente.get(m.getClientID());
            if (topics != null) {
                topics.remove(m.getTopicFilter());
            }
        }

        @Override
        public void onDisconnect(InterceptDisconnectMessage m) {
            topicsPorCliente.remove(m.getClientID());
        }

        @Override
        public void onConnectionLost(InterceptConnectionLostMessage m) {
            topicsPorCliente.remove(m.getClientID());
        }
    }
}
```

`puertoLibre()` : abre un socket en el puerto 0 (el sistema operativo asigna uno libre), toma el número y lo cierra. Así varios procesos de test no chocan.

`Espia` : interceptor de Moquette que anota qué cliente se suscribe a qué topic. Es clase interna para acceder al mapa.

Nombre del interceptor (Moquette lo exige).

Errores internos del bucle de sesión: no interesan para los tests.

Al suscribirse, agrega el topic al conjunto de ese cliente (lo crea si no existía).

Al desuscribirse, saca el topic. Sirve para verificar que el engine deja de escuchar un termostato que se borró o cambió de topic.

Desconexión ordenada: se olvidan todas las suscripciones de ese cliente (por ejemplo, el engine de un test anterior que se cerró).

Conexión perdida (sin desconexión ordenada): mismo tratamiento.

Cierre de la clase interna y de la clase.

5. Switches HTTP falsos

modules/engine/src/test/java/uy/edu/utec/iiss/engine/integracion/SwitchesHttpFalsos.java — líneas 1–134

```java
package uy.edu.utec.iiss.engine.integracion;
```

Mismo paquete `integracion`.

```java

```

```java
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
```

Jackson para leer el cuerpo JSON de los comandos.

```java
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
```

El servidor HTTP que trae el JDK (`com.sun.net.httpserver`): no agrega dependencias.

```java
import java.io.IOException;
import java.io.UncheckedIOException;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.Executors;
```

Imports de E/S, red, charset y colecciones concurrentes.

```java

/**
 * Servidor HTTP de switches que habla el estándar v2 de verdad (§3 {@code POST
 * /switches/{id}/comandos}, §4 {@code GET /switches/{id}}), así el engine usa su {@code
 * ClienteSwitchRest} real. Cada switch recuerda su estado; se puede fijar el estado inicial (la
 * calefacción que ya estaba encendida) o hacer que un switch deje de responder.
 */
```

Javadoc: es un servidor HTTP que implementa de verdad los endpoints de switches del estándar (§3 comandar, §4 consultar). Así el engine usa su `ClienteSwitchRest` real, con red, timeouts y JSON incluidos.

```java
public final class SwitchesHttpFalsos {
```

Clase `final` de utilidad.

```java

    /** Un comando recibido, en orden de llegada. */
    public record Comando(String idSwitch, String accion) {}
```

`Comando`: un comando recibido (switch y acción). La lista de comandos queda en orden de llegada; con ella se reconstruye el pico de potencia.

```java

    private static final ObjectMapper JSON = new ObjectMapper();
```

Un único `ObjectMapper` compartido (es thread-safe).

```java

    private final HttpServer server;
```

El servidor HTTP.

```java
    private final Map<String, String> estados = new ConcurrentHashMap<>();
```

Estado actual de cada switch ("ON"/"OFF").

```java
    private final Set<String> caidos = ConcurrentHashMap.newKeySet();
```

Switches "caídos": responden 503 a todo. Lo va a usar la Persona 3 para la falla REST.

```java
    private final List<Comando> comandos = new CopyOnWriteArrayList<>();
    private final List<String> consultas = new CopyOnWriteArrayList<>();
```

Historial de comandos (POST) y de consultas (GET), en listas seguras para escribir desde varios hilos.

```java

```

```
    public SwitchesHttpFalsos() {
        try {
            server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
```

El constructor crea el servidor en 127.0.0.1 y en un puerto libre (el 0 deja que lo elija el sistema operativo).

```
        server.createContext("/switches", this::atender);
```

Todas las rutas que empiezan con `/switches` las atiende `atender`.

```
        server.setExecutor(Executors.newCachedThreadPool());
```

Un pool de hilos, así un pedido lento no bloquea a los demás.

```
        server.start();
    }

    public String url() {
        return "http://127.0.0.1:" + server.getAddress().getPort();
    }

    /** Vuelve a cero: sin estados, sin historial, todos responden. */
    public void reiniciar() {
        estados.clear();
        caidos.clear();
        comandos.clear();
        consultas.clear();
    }

    /** Estado del switch; {@code "OFF"} si nunca se tocó. */
    public String estado(String idSwitch) {
        return estados.getOrDefault(idSwitch, "OFF");
    }

    /** Fija el estado sin pasar por el controlador (estado inicial, cambio manual). */
    public void fijar(String idSwitch, String estado) {
        estados.put(idSwitch, estado);
    }

    /** {@code true}: el switch contesta 503 a todo; {@code false}: vuelve a responder. */
    public void caido(String idSwitch, boolean caido) {
        if (caido) {
            caidos.add(idSwitch);
        } else {
            caidos.remove(idSwitch);
        }
    }

    public List<Comando> comandos() {
        return List.copyOf(comandos);
    }

    public List<Comando> comandosA(String idSwitch) {
        return comandos.stream().filter(c -> c.idSwitch().equals(idSwitch)).toList();
    }
```

Arranca el servidor.

URL base que se pone en `urlSwitch` del inventario.

`reiniciar()` : vuelve a cero entre tests (sin estados, sin caídos, sin historial).

Estado de un switch; si nunca se tocó, "OFF" (calefacción apagada).

`fijar()` : cambia el estado sin pasar por el controlador. Sirve para simular la calefacción que ya estaba encendida al arrancar, o que alguien la tocó a mano.

`caido(id, true/false)` : hace que un switch deje de responder o vuelva a responder.

Copia inmutable del historial de comandos.

Solo los comandos dirigidos a un switch.

```
86
87      public List<String> consultas() {
88          return List.copyOf(consultas);
89      }
90
91      public void detener() {
92          server.stop(0);
93      }
94
95      private void atender(HttpExchange ex) throws IOException {
96          try (ex) {
97              // /switches/{id} o /switches/{id}/comandos
98              String[] partes = ex.getRequestURI().getPath().split("/");
99              if (partes.length < 3) {
100                 responder(ex, 404, "{}");
101                 return;
102             }
103             String id = partes[2];
104             if (caidos.contains(id)) {
105                 responder(ex, 503, "{\"mensaje\":\"switch caído\"}");
106                 return;
107             }
108             boolean esComando = partes.length == 4 && partes[3].equals("comandos");
109             if (esComando && ex.getRequestMethod().equals("POST")) {
110                 JsonNode cuerpo = JSON.readTree(ex.getRequestBody());
111                 String accion = cuerpo.path("accion").asText();
112                 if (!accion.equals("ON") && !accion.equals("OFF")) {
113                     responder(ex, 400, "{\"mensaje\":\"accion inválida\"}");
114                     return;
115                 }
116                 estados.put(id, accion);
117                 comandos.add(new Comando(id, accion));
118             } else if (partes.length == 3 && ex.getRequestMethod().equals("GET")) {
119                 consultas.add(id);
120             } else {
121                 responder(ex, 405, "{}");
122                 return;
123             }
124             responder(ex, 200, "{\"id\":\"%s\",\"estado\":\"%s\"}".formatted(id, estado(id)));
125         }
126     }
127
```

Copia del historial de consultas (GET).

Baja el servidor.

`atender()` : maneja cada pedido; el try-with-resources cierra el intercambio HTTP al terminar.

Separa la ruta en partes: `/switches/{id}` → ["", "switches", id]; `/switches/{id}/comandos` agrega "comandos".

Ruta incompleta → 404.

El id del switch está en la tercera parte.

Si el switch está "caído", contesta 503 (servicio no disponible) sin cambiar nada.

Es un comando si la ruta termina en `/comandos` .

POST de comando: lee el JSON y toma el campo `accion` .

Si la acción no es ON ni OFF, responde 400, como un switch real.

Aplica la acción al estado y la anota en el historial.

GET `/switches/{id}` : es una consulta (§4); se anota.

Cualquier otra combinación de método y ruta → 405.

Respuesta 200 con `{"id":…,"estado":…}` , el formato que espera `ClienteSwitchRest` .

Cierre del try y del método.

```
    private static void responder(HttpExchange ex, int status, String cuerpo) throws IOException {
        byte[] bytes = cuerpo.getBytes(StandardCharsets.UTF_8);
        ex.getResponseHeaders().set("Content-Type", "application/json");
        ex.sendResponseHeaders(status, bytes.length);
        ex.getResponseBody().write(bytes);
    }
}
```

`responder()` : pasa el cuerpo a bytes UTF-8, pone `Content-Type: application/json`, manda el código y el largo exacto, y escribe el cuerpo.

Fin de la clase.

6. Clase base de los tests de integración

modules/engine/src/test/java/uy/edu/utec/iiss/engine/integracion/IntegracionBase.java — líneas 1–297

```java
package uy.edu.utec.iiss.engine.integracion;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.junit.jupiter.api.Assertions.fail;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import com.fasterxml.jackson.databind.ObjectMapper;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Duration;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.function.BooleanSupplier;
import java.util.function.Supplier;
import java.util.stream.Collectors;

import org.junit.jupiter.api.BeforeEach;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.annotation.DirtiesContext;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.request.MockHttpServletRequestBuilder;

import uy.edu.utec.iiss.engine.adaptador.CoreAdapter;
import uy.edu.utec.iiss.engine.adaptador.CoreAdapter.EstadoControlado;
import uy.edu.utec.iiss.engine.inventario.Inventario;

```

Paquete `integracion`.

Imports estáticos de JUnit (aserciones) y de MockMvc (para armar pedidos PUT/POST y verificar el status).

Imports del JDK: JSON (Jackson), archivos temporales, duraciones, colecciones e interfaces funcionales para las condiciones de espera.

Imports de Spring Test: contexto completo de Spring Boot, MockMvc, propiedades dinámicas y `DirtiesContext`.

Clases del engine que se usan para verificar: el `CoreAdapter` y el inventario.

```java
/**
 * Base de los tests de integración: el engine completo (Spring, listener MQTT, core, cliente REST
 * de switches) contra un broker MQTT embebido y switches HTTP falsos, sin Docker.
 *
 * <ul>
 *   <li>Los estímulos entran como en producción: el sitio y el control por el API REST (§5), las
 *       lecturas por MQTT (§2).
 *   <li>Se verifica lo que ven los switches (§3/§4) y, para explicar el porqué, el motivo de la
 *       decisión en el {@link CoreAdapter}.
 *   <li>Cada test arranca un engine nuevo; el broker y los switches se comparten y se limpian.
 * </ul>
 *
 * Por defecto el control arranca el jueves 2026-10-01 a las 09:00 UTC, fuera de la franja punta.
 */
```

Javadoc: qué se arma (engine completo contra un broker embebido y switches HTTP falsos, sin Docker), cómo entran los estímulos (REST y MQTT, como en producción), qué se verifica y la fecha por defecto del control.

```java
@SpringBootTest(
        properties = {
            "api.security.api-key=" + IntegracionBase.CLAVE,
            "entorno.url=",
            "control.zona-horaria=UTC",
            "control.tick-ms=100",
            "control.reconciliacion-segundos=1",
            "switch.espera-reintento-segundos=1",
            "switch.timeout-conexion-ms=500",
            "switch.timeout-lectura-ms=1000"
        })
@AutoConfigureMockMvc
@DirtiesContext(classMode = DirtiesContext.ClassMode.AFTER_EACH_TEST_METHOD)
```

`@SpringBootTest`: levanta el engine entero, con todos sus beans reales.

Propiedades fijas para todos los tests. La API key del controlador se toma de la constante `CLAVE`.

Sin entorno (simulador): el engine usa directamente `mqtt.broker.url`, que apunta al broker embebido.

La franja tarifaria se lee en UTC, así 09:00Z está fuera de punta y 18:00Z adentro.

Tick del control cada 100 ms (en producción, 1 s): los tests no tienen que esperar tanto.

Reconciliación de switches cada 1 s (en producción, 15 s).

Un switch que falla se reintenta al segundo.

Timeouts HTTP cortos hacia los switches (0.5 s para conectar, 1 s para leer).

Cierre de las propiedades.

`@AutoConfigureMockMvc`: permite hacer pedidos al API REST del engine sin abrir un puerto.

`@DirtiesContext` después de cada test: cada test arranca con un engine nuevo (control detenido, sin lecturas, sin decisiones). Así ningún test depende de otro.

```java
public abstract class IntegracionBase {

    static final String CLAVE = "clave-it";
    static final String FECHA_INICIO = "2026-10-01T09:00:00Z";
    static final double POTENCIA_SITIO = 3.7;
    static final Duration ESPERA = Duration.ofSeconds(5);

```

Clase abstracta: no se ejecuta sola, la heredan las clases de test.

API key que usan todos los pedidos.

Fecha por defecto de INICIAR: jueves 2026-10-01 a las 09:00 UTC (día hábil, fuera de la franja 17–23).

Potencia contratada por defecto, la del sitio de ejemplo del estándar (3.7 kW).

Tiempo máximo que se espera a que algo ocurra antes de fallar el test.

```
    protected static final BrokerMqttEmbebido BROKER = new BrokerMqttEmbebido();
    protected static final SwitchesHttpFalsos SWITCHES = new SwitchesHttpFalsos();

    @DynamicPropertySource
    static void entorno(DynamicPropertyRegistry r) throws Exception {
        BROKER.iniciar();
        Path archivo = Files.createTempDirectory("ecowarm-it").resolve("inventario.json");
        r.add("inventario.archivo", archivo::toString);
        r.add("mqtt.broker.url", BROKER::url);
    }

    @Autowired protected MockMvc mvc;
    @Autowired protected CoreAdapter adaptador;
    @Autowired protected ObjectMapper json;

    @BeforeEach
    void limpiarEntorno() {
        BROKER.iniciar(); // por si un test lo dejó caído
        SWITCHES.reiniciar();
    }

    // ---------------------------------------------------------------------
    // Inventario
    // ---------------------------------------------------------------------

    protected static String idSwitch(String idHabitacion) {
        return "sw-" + idHabitacion;
    }

    protected static String topic(String idHabitacion) {
        return "ht-" + idHabitacion + "/status/temperature:0";
    }

    protected static Inventario.Habitacion habitacion(
            String id, double temperaturaEsperada, double potenciaKW) {
        return new Inventario.Habitacion(
                id,
                "Hab " + id,
                temperaturaEsperada,
                potenciaKW,
                "ht-" + id,
                topic(id),
                idSwitch(id),
                SWITCHES.url());
    }
```

Un broker y un servidor de switches para toda la corrida (`static`), compartidos por todos los tests; se limpian antes de cada uno.

`@DynamicPropertySource` : Spring lo ejecuta antes de crear el contexto, para calcular propiedades en tiempo de ejecución.

Garantiza que el broker esté levantado antes de que arranque el engine.

El inventario se guarda en un archivo temporal nuevo, así no se pisa con `./data/inventario.json` ni entre tests.

El engine se conecta al broker embebido.

Inyecta MockMvc (pedidos REST), el `CoreAdapter` real (para leer motivos y temperaturas) y el `ObjectMapper` de Spring (para serializar el inventario).

Antes de cada test: vuelve a levantar el broker por si un test lo dejó caído y deja los switches en cero.

Sección de helpers para armar el inventario.

Convención de nombres: el switch de la habitación "living" es "sw-living".

Topic del termostato con el formato del simulador: `ht-living/status/temperature:0` .

`habitacion(id, esperada, potencia)` : arma una habitación completa del inventario v2 (nombre, termostato, topic, switch) apuntando al servidor de switches falso.

```
    protected static Inventario inventario(double potenciaKW, Inventario.Habitacion... habs) {
        return new Inventario(
                "2.0",
                new Inventario.Sitio(
                        "sitio-it",
                        "Sitio de integración",
                        potenciaKW,
                        new Inventario.Tarifa(new Inventario.Franja("17:00", "23:00", "HABILES"))),
                List.of(habs));
    }
```

`inventario(potencia, habitaciones…)` : arma el sitio con id fijo, la potencia indicada y la franja punta 17:00–23:00 en días hábiles.

```
    /** PUT /sitio (§5.1). */
    protected void cargarSitio(Inventario inv) throws Exception {
        mvc.perform(conClave(put("/sitio")).content(json.writeValueAsString(inv)))
                .andExpect(status().isOk());
    }
```

`cargarSitio(inv)` : hace PUT /sitio (§5.1) con el inventario en JSON y verifica que responda 200.

```
    protected void cargarSitio(double potenciaKW, Inventario.Habitacion... habs) throws Exception {
        cargarSitio(inventario(potenciaKW, habs));
    }
```

Variante corta: arma el inventario y lo carga.

```
    /** PUT /habitaciones/{id} (§5.1). */
    protected void modificarHabitacion(Inventario.Habitacion h) throws Exception {
        mvc.perform(conClave(put("/habitaciones/" + h.id())).content(json.writeValueAsString(h)))
                .andExpect(status().isOk());
    }
```

`modificarHabitacion(h)` : PUT /habitaciones/{id} (§5.1), para cambios en caliente.

```
    // ---------------------------------------------------------------------
    // Control (§5.2)
    // ---------------------------------------------------------------------
```

Sección de helpers del control.

```
    protected void iniciar() throws Exception {
        iniciar(FECHA_INICIO, 1);
    }
```

`iniciar()` : INICIAR con la fecha por defecto y sin aceleración (factor 1).

```
    protected void iniciar(String fechaHora, double factorTiempo) throws Exception {
        comando(
                "{\"accion\":\"INICIAR\",\"fechaHora\":\"%s\",\"factorTiempo\":%s}"
                        .formatted(fechaHora, factorTiempo));
    }
```

`iniciar(fechaHora, factor)` : arma el cuerpo `{"accion":"INICIAR","fechaHora":…,"factorTiempo":…}` del §5.2.

```
    protected void detener() throws Exception {
        comando("{\"accion\":\"DETENER\"}");
    }
```

`detener()` : manda `{"accion":"DETENER"}`.

```
    private void comando(String cuerpo) throws Exception {
        mvc.perform(conClave(post("/control/comandos")).content(cuerpo)).andExpect(status().isOk());
    }
```

Envía el comando a POST /control/comandos y verifica que responda 200.

```
    protected static MockHttpServletRequestBuilder conClave(MockHttpServletRequestBuilder b) {
        return b.header("X-API-Key", CLAVE).contentType(MediaType.APPLICATION_JSON);
    }
```

`conClave()` : agrega el header `X-API-Key` y `Content-Type: application/json` a cualquier pedido.

```
    // ---------------------------------------------------------------
    // Termostatos (§2)
    // ---------------------------------------------------------------
```

Sección de helpers de los termostatos.

```
    /**
     * Publica la lectura cuando el engine ya está suscrito al topic y, con el control en ejecución,
     * espera a que el core la registre.
     */
```

Javadoc de `temperatura()` : el problema que resuelve es que una lectura publicada antes de que el engine esté suscrito se pierde.

```
    protected void temperatura(String idHabitacion, double temperaturaC) {
```

`temperatura(habitación, °C)` : simula que el termostato manda una lectura.

```
        esperar(
                () -> BROKER.haySuscriptor(topic(idHabitacion)),
                () -> "el engine no se suscribió a " + topic(idHabitacion));
```

Primero espera a que el engine esté suscrito al topic de esa habitación (lo informa el broker).

```
        BROKER.publicarTemperatura(topic(idHabitacion), temperaturaC);
```

Publica la lectura por MQTT.

```
        if (adaptador.habilitado()) {
            esperar(
                    () -> Objects.equals(estado(idHabitacion).temperaturaActual(), temperaturaC),
                    () -> "el core no registró " + temperaturaC + " °C en " + idHabitacion);
        }
```

Si el control está en ejecución, espera a que el core la haya registrado (temperatura actual igual a la publicada). Así, cuando la función vuelve, la decisión ya se tomó y el test no depende de la velocidad de la máquina. Si está detenido, el engine la ignora a propósito y no hay nada que esperar.

```
    }
```

Fin de `temperatura()` .

```
    // ---------------------------------------------------------------
    // Verificaciones
    // ---------------------------------------------------------------
```

Sección de verificaciones.

```
    protected EstadoControlado estado(String idHabitacion) {
        return adaptador.estados().stream()
                .filter(e -> e.idHabitacion().equals(idHabitacion))
                .findFirst()
                .orElseThrow(() -> new AssertionError("no existe la habitación " + idHabitacion));
    }
```

`estado(habitación)` : busca la foto de diagnóstico de la habitación en el `CoreAdapter` (temperatura, decisión, motivo, estado del switch); si no existe, falla con un mensaje claro.

```
    protected String motivo(String idHabitacion) {
        return estado(idHabitacion).motivo();
    }
```

`motivo(habitación)` : el porqué de la última decisión (CALEFACCIONANDO, SATISFECHA, SIN_LECTURA, SIN_POTENCIA_DISPONIBLE, EXCEDE_CONTRATADA, TARIFA_PUNTA…).

```
    /** Espera a que el switch de la habitación quede en {@code estado} ("ON"/"OFF"). */
    protected void esperarSwitch(String idHabitacion, String estado) {
```

`esperarSwitch(habitación, "ON"/"OFF")` : espera hasta que el switch falso quede en ese estado. Mira lo que recibió el switch, no lo que el engine cree.

```
        esperar(
                () -> SWITCHES.estado(idSwitch(idHabitacion)).equals(estado),
```

Condición: el estado del switch coincide.

```
                () ->
                        "el switch de %s quedó %s y se esperaba %s (motivo: %s); comandos: %s"
                                .formatted(
                                        idHabitacion,
                                        SWITCHES.estado(idSwitch(idHabitacion)),
                                        estado,
                                        motivo(idHabitacion),
                                        SWITCHES.comandos()));
    }

    /** Espera a que el core decida por el motivo dado (p. ej. "SIN_POTENCIA_DISPONIBLE"). */
    protected void esperarMotivo(String idHabitacion, String motivo) {
        esperar(
                () -> motivo.equals(motivo(idHabitacion)),
                () -> idHabitacion + " tiene motivo " + motivo(idHabitacion) + ", no " + motivo);
    }

    /** Verifica que la condición se mantiene durante todo el lapso (lo que NO tiene que pasar). */
    protected static void seMantiene(
            Duration lapso, BooleanSupplier condicion, Supplier<String> mensaje) {

        long limite = System.nanoTime() + lapso.toNanos();
        while (System.nanoTime() < limite) {
            assertTrue(condicion.getAsBoolean(), mensaje);
            dormir(50);
        }
    }

    /** kW encendidos según lo que reportan los switches. */
    protected static double potenciaEncendida(Inventario inv) {
        return inv.habitaciones().stream()
                .filter(h -> SWITCHES.estado(h.idSwitch()).equals("ON"))
                .mapToDouble(Inventario.Habitacion::potenciaKW)
                .sum();
    }

    /**
     * Mayor potencia que llegó a estar encendida a la vez, reproduciendo los comandos en el orden
     * en que los recibieron los switches (todos arrancan apagados). Detecta excesos transitorios
     * que el estado final no muestra.
     */
    protected static double picoDePotencia(Inventario inv) {

        Map<String, Double> potencia =
                inv.habitaciones().stream()
                        .collect(
                                Collectors.toMap(
                                        Inventario.Habitacion::idSwitch,
                                        Inventario.Habitacion::potenciaKW));
```

Si se vence el tiempo, el mensaje de falla incluye el estado real, el esperado, el motivo del core y todos los comandos recibidos, para diagnosticar sin volver a correr.

Fin de `esperarSwitch()`.

`esperarMotivo(habitación, motivo)`: espera a que el core decida por esa razón.

`seMantiene(lapso, condición, mensaje)`: lo contrario de esperar. Verifica que algo NO pase durante un lapso (por ejemplo, que no se mande ningún comando con el control detenido).

Revisa la condición cada 50 ms hasta que se cumple el lapso; si en algún momento es falsa, falla.

`potenciaEncendida(inv)`: suma los kW de las habitaciones cuyo switch está en ON en este momento.

Javadoc de `picoDePotencia`: el estado final puede estar bien y aun así haber habido un instante con más carga que la contratada; esto lo detecta.

`picoDePotencia(inv)`: mayor carga simultánea que hubo.

Mapa switch → kW, tomado del inventario.

```
        Map<String, Boolean> encendido = new HashMap<>();
        double pico = 0;
```

Estado reconstruido de cada switch y pico máximo, ambos empiezan en cero (todo apagado).

```
        for (SwitchesHttpFalsos.Comando c : SWITCHES.comandos()) {
            encendido.put(c.idSwitch(), c.accion().equals("ON"));
```

Recorre los comandos en el orden en que los recibieron los switches y aplica cada uno.

```
            double ahora =
                    encendido.entrySet().stream()
                            .filter(Map.Entry::getValue)
                            .mapToDouble(e -> potencia.getOrDefault(e.getKey(), 0.0))
                            .sum();
```

Después de cada comando, suma los kW de los switches encendidos.

```
            pico = Math.max(pico, ahora);
        }
```

Se queda con el máximo. Este helper fue el que detectó el bug del orden ON/OFF.

```
        return pico;
    }

    protected static Map<String, String> estadosSwitches(Inventario inv) {
        return inv.habitaciones().stream()
                .collect(
                        Collectors.toMap(
                                Inventario.Habitacion::id, h -> SWITCHES.estado(h.idSwitch())));
    }

    protected static void esperar(BooleanSupplier condicion, Supplier<String> mensaje) {
        long limite = System.nanoTime() + ESPERA.toNanos();
        while (!condicion.getAsBoolean()) {
            if (System.nanoTime() > limite) {
                fail(mensaje.get());
            }
            dormir(20);
        }
    }

    protected static void assertSwitch(String idHabitacion, String estado) {
        assertEquals(estado, SWITCHES.estado(idSwitch(idHabitacion)), "switch de " + idHabitacion);
    }

    private static void dormir(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new IllegalStateException(e);
        }
    }
}
```

Devuelve el pico.

`estadosSwitches(inv)` : mapa habitación → "ON"/"OFF", para comparar el estado completo de una vez o mostrarlo en un mensaje de error.

`esperar(condición, mensaje)` : revisa la condición cada 20 ms; si pasa el tiempo `ESPERA` (5 s) sin cumplirse, falla con el mensaje. El mensaje se calcula recién al fallar.

`assertSwitch(habitación, estado)` : verificación inmediata, sin espera, del estado de un switch.

`dormir(ms)` : `Thread.sleep` sin checked exception; si el hilo se interrumpe, restaura la marca de interrupción y corta.

Fin de la clase.

7. Tests: control según la carga total

modules/engine/src/test/java/uy/edu/utec/iiss/engine/integracion/ControlCargaTest.java — líneas 1–156

```java
package uy.edu.utec.iiss.engine.integracion;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.time.Duration;
import java.util.Map;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import uy.edu.utec.iiss.engine.inventario.Inventario;

```

Paquete e imports: aserciones de JUnit, `Duration`, `Map`, anotaciones de JUnit y el inventario.

```java
/**
 * Control de temperatura de punta a punta: lecturas por MQTT, decisiones del core y comandos REST a
 * los switches. Primero con potencia de sobra y después con la potencia contratada como límite
 * efectivo.
 */
```

Javadoc: control de punta a punta (MQTT → core → REST), primero con potencia de sobra y después con la potencia contratada como límite efectivo.

```java
@DisplayName("Integración: control de temperatura según la carga total")
class ControlCargaTest extends IntegracionBase {

```

Nombre que se ve en el reporte y herencia de `IntegracionBase` (con eso ya hay engine, broker y switches).

```java
    @Test
    @DisplayName(
            "Carga no limitante: se encienden todas las que tienen frío y cada una se apaga al"
                    + " llegar a su temperatura")
```

Test 1. Caso de la consigna: control de temperatura cuando la carga total no es una limitación.

```java
    void cargaNoLimitante() throws Exception {
        // 3 × 1.0 kW = 3.0 kW < 3.7 kW contratados
```

Tres habitaciones de 1.0 kW suman 3.0 kW, menos que los 3.7 kW contratados.

```java
        cargarSitio(
                POTENCIA_SITIO,
                habitacion("living", 21.0, 1.0),
                habitacion("dormitorio", 20.0, 1.0),
                habitacion("estudio", 19.0, 1.0));
```

Carga el sitio con living (esperada 21 °C), dormitorio (20 °C) y estudio (19 °C).

```java
        iniciar();

```

INICIAR a las 09:00 de un jueves (fuera de punta).

```java
        temperatura("living", 18.0);
        temperatura("dormitorio", 17.5);
        temperatura("estudio", 16.0);
```

Las tres reportan temperaturas por debajo de su esperada.

```java
        esperarSwitch("living", "ON");
        esperarSwitch("dormitorio", "ON");
        esperarSwitch("estudio", "ON");

```

Como la potencia alcanza, se encienden las tres.

```java
        temperatura("dormitorio", 20.1);
        esperarSwitch("dormitorio", "OFF");
```

El dormitorio llega a 20.1 °C (pasa su esperada de 20) y se apaga.

```java
        assertEquals("SATISFECHA", motivo("dormitorio"));
```

El motivo es SATISFECHA: está en su temperatura.

```
        assertSwitch("living", "ON");
        assertSwitch("estudio", "ON");
```

Las otras dos siguen encendidas: apagar una no afecta a las demás.

```

```

```
        temperatura("living", 21.0);
        temperatura("estudio", 19.5);
        esperarSwitch("living", "OFF");
        esperarSwitch("estudio", "OFF");
```

Living y estudio llegan a su temperatura y también se apagan.

```
    }
```

Fin del test 1.

```

```

```
    @Test
    @DisplayName("Carga no limitante: una habitación ya caliente no se enciende")
    void habitacionCalienteNoSeEnciende() throws Exception {
```

Test 2. Una habitación que ya está caliente no se enciende nunca.

```
        cargarSitio(
                POTENCIA_SITIO, habitacion("living", 21.0, 1.0), habitacion("cocina", 19.0, 1.0));
        iniciar();
```

Living (esperada 21) y cocina (esperada 19), y arranca el control.

```

```

```
        temperatura("living", 18.0);
        temperatura("cocina", 22.0);
```

Living tiene frío (18) y la cocina está a 22, por encima de su esperada.

```
        esperarSwitch("living", "ON");
        esperarMotivo("cocina", "SATISFECHA");
```

Living se enciende; la cocina queda SATISFECHA.

```
        seMantiene(
                Duration.ofMillis(500),
                () ->
                        SWITCHES.comandosA(idSwitch("cocina")).stream()
                                .noneMatch(c -> c.accion().equals("ON")),
                () -> "se encendió la cocina, que ya estaba caliente");
```

Durante medio segundo se verifica que el switch de la cocina no reciba ningún ON.

```
    }
```

Fin del test 2.

```

```

```
    @Test
    @DisplayName(
            "Carga limitante: con 3 × 1.5 kW y 3.7 kW contratados se encienden solo las dos de"
                    + " mayor déficit")
```

Test 3. Caso de la consigna: la carga total es una limitación efectiva.

```
    void cargaLimitante() throws Exception {
        Inventario inv =
                inventario(
                        POTENCIA_SITIO,
                        habitacion("living", 22.0, 1.5),
                        habitacion("dormitorio", 22.0, 1.5),
                        habitacion("estudio", 22.0, 1.5));
        cargarSitio(inv);
```

Tres habitaciones de 1.5 kW (4.5 kW en total) con 3.7 kW contratados: entran solo dos.

```
        iniciar();
```

Arranca el control.

```

```

```
        temperatura("living", 16.0); // déficit 6
        temperatura("dormitorio", 17.0); // déficit 5
        temperatura("estudio", 20.0); // déficit 2
```

Déficits de 6, 5 y 2 °C. El criterio del core es "mayor déficit primero".

```
        esperarSwitch("living", "ON");
        esperarSwitch("dormitorio", "ON");
```

Se encienden living y dormitorio, las de mayor déficit (3.0 kW).

```
        esperarMotivo("estudio", "SIN_POTENCIA_DISPONIBLE");
        assertSwitch("estudio", "OFF");
```

El estudio queda apagado con motivo SIN_POTENCIA_DISPONIBLE.

```
        seMantiene(
                Duration.ofMillis(500),
                () -> potenciaEncendida(inv) <= POTENCIA_SITIO,
                () -> "se superó la potencia contratada: " + estadosSwitches(inv));
```

Durante medio segundo, la potencia encendida nunca supera la contratada.

```
        assertTrue(picoDePotencia(inv) <= POTENCIA_SITIO, "comandos: " + SWITCHES.comandos());
```

Y en todo el historial de comandos tampoco hubo un pico por encima de 3.7 kW.

```
    }
```

Fin del test 3.

```

    @Test
    @DisplayName(
            "Carga limitante: cuando una se satisface, la potencia liberada pasa a la que esperaba,"
                    + " sin superar nunca la contratada")
```

Test 4. Cuando una habitación se satisface, la potencia que libera pasa a la que esperaba, sin superar nunca la contratada.

```
    void cargaLimitante_redistribuye() throws Exception {
        // La que espera va primera en el inventario: si el engine mandara su ON antes del OFF de
        // la que se satisface, habría un instante con 4.5 kW encendidos.
```

El comentario explica la trampa: la habitación que espera va primera en el inventario. Antes del fix, el engine mandaba su ON antes del OFF de la que se satisfacía.

```
        Inventario inv =
                inventario(
                        POTENCIA_SITIO,
                        habitacion("estudio", 22.0, 1.5),
                        habitacion("living", 22.0, 1.5),
                        habitacion("dormitorio", 22.0, 1.5));
        cargarSitio(inv);
```

Estudio, living y dormitorio, de 1.5 kW cada una, con 3.7 kW contratados.

```
        iniciar();
        temperatura("living", 16.0);
        temperatura("dormitorio", 17.0);
        temperatura("estudio", 20.0);
```

Arranca el control. Living (déficit 6) y dormitorio (déficit 5) entran; el estudio (déficit 2) espera.

```
        esperarSwitch("living", "ON");
        esperarSwitch("dormitorio", "ON");

```

Confirma que están encendidas las dos de mayor déficit.

```
        temperatura("living", 22.0);
        esperarSwitch("living", "OFF");
```

Living llega a 22 °C y se apaga.

```
        esperarSwitch("estudio", "ON");
        assertSwitch("dormitorio", "ON");
```

Su potencia pasa al estudio, que se enciende; el dormitorio sigue encendido.

```
        assertEquals(
                Map.of("living", "OFF", "dormitorio", "ON", "estudio", "ON"), estadosSwitches(inv));
```

El estado final completo es el esperado.

```
        assertTrue(
                picoDePotencia(inv) <= POTENCIA_SITIO,
                "en algún momento se superó la potencia contratada; comandos: "
                        + SWITCHES.comandos());
```

La verificación clave: reconstruyendo el orden real de los comandos, nunca hubo más de 3.7 kW encendidos. Antes del fix este assert fallaba con un pico de 4.5 kW.

```
    }
```

Fin del test 4.

```

    @Test
    @DisplayName("Carga limitante: una habitación que sola supera la contratada nunca se enciende")
    void habitacionQueExcedeLaContratada() throws Exception {
```

Test 5. Una habitación cuya potencia sola supera la contratada nunca se enciende.

```
        cargarSitio(
                POTENCIA_SITIO, habitacion("galpon", 20.0, 4.0), habitacion("living", 21.0, 1.0));
        iniciar();
```

"galpon" consume 4.0 kW (más que los 3.7 contratados); living, 1.0 kW.

```

```

```
        temperatura("galpon", 10.0);
        temperatura("living", 18.0);
        esperarSwitch("living", "ON");
```

Las dos tienen frío; living se enciende normalmente.

```
        esperarMotivo("galpon", "EXCEDE_CONTRATADA");
        assertSwitch("galpon", "OFF");
```

El galpón queda apagado con motivo EXCEDE_CONTRATADA: encenderlo haría saltar la llave.

```
    }
```

Fin del test 5.

```

```

```
    @Test
    @DisplayName("En la franja punta no se calefacciona aunque sobre potencia")
    void franjaPunta() throws Exception {
```

Test 6. En la franja punta no se calefacciona aunque sobre potencia.

```
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar("2026-10-01T18:00:00Z", 1); // jueves 18:00, dentro de 17:00–23:00
```

INICIAR a las 18:00 de un jueves, dentro de la franja 17:00–23:00.

```

```

```
        temperatura("living", 15.0);
        esperarMotivo("living", "TARIFA_PUNTA");
        assertSwitch("living", "OFF");
```

Aunque la habitación está a 15 °C, queda apagada con motivo TARIFA_PUNTA.

```
    }
}
```

Fin del test y de la clase.

8. Tests: situaciones iniciales y arranque / parada

modules/engine/src/test/java/uy/edu/utec/iiss/engine/integracion/ArranqueParadaTest.java — líneas 1–184

```java
package uy.edu.utec.iiss.engine.integracion;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.time.Duration;
import java.util.List;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

/**
 * Situaciones iniciales y arranque / parada del controlador (§5.2): qué hace con la calefacción que
 * encuentra al INICIAR y qué deja al DETENER.
 */
@DisplayName("Integración: situaciones iniciales y arranque / parada")
class ArranqueParadaTest extends IntegracionBase {

    // ---------------- situaciones iniciales ----------------

    @Test
    @DisplayName("Sin INICIAR no se toca ningún switch, aunque lleguen lecturas")
    void sinIniciarNoSeComanda() throws Exception {
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        temperatura("living", 15.0);

        seMantiene(
                Duration.ofMillis(700),
                () -> SWITCHES.comandos().isEmpty() && SWITCHES.consultas().isEmpty(),
                () -> "se habló con los switches sin INICIAR: " + SWITCHES.comandos());
    }

    @Test
    @DisplayName("Al INICIAR consulta el estado real de cada switch (§4) antes de comandar")
    void iniciarConsultaLosSwitches() throws Exception {
        cargarSitio(
                POTENCIA_SITIO, habitacion("living", 21.0, 1.0), habitacion("cocina", 19.0, 1.0));
        iniciar();

        esperar(
                () -> SWITCHES.consultas().containsAll(List.of("sw-living", "sw-cocina")),
                () -> "no consultó todos los switches: " + SWITCHES.consultas());
    }

```

Paquete e imports.

Javadoc: qué hace el controlador con la calefacción que encuentra al INICIAR y qué deja al DETENER (§5.2).

Nombre en el reporte y herencia de la base.

Primer grupo: situaciones iniciales.

Test 1. Sin INICIAR, el engine no habla con ningún switch aunque lleguen lecturas.

Carga el sitio y publica una lectura fría, sin iniciar el control.

Durante 0.7 s no hubo ni comandos ni consultas a los switches.

Fin del test 1.

Test 2. Al INICIAR, el engine consulta el estado real de cada switch (§4) antes de comandar.

Dos habitaciones y se inicia el control.

Espera a que los dos switches hayan recibido su GET de consulta.

Fin del test 2.

```
    @Test
    @DisplayName(
            "Calefacción encendida de antes y todavía sin lecturas: se apaga al INICIAR (no se"
                    + " calefacciona a ciegas)")
    void encendidaSinLecturas_seApaga() throws Exception {
        SWITCHES.fijar("sw-living", "ON");
        SWITCHES.fijar("sw-cocina", "ON");

        cargarSitio(
                POTENCIA_SITIO, habitacion("living", 21.0, 1.0), habitacion("cocina", 19.0, 1.0));
        iniciar();

        esperarSwitch("living", "OFF");
        esperarSwitch("cocina", "OFF");
        assertEquals("SIN_LECTURA", motivo("living"));
    }

    @Test
    @DisplayName(
            "Calefacción encendida de antes en una habitación fría y otra caliente: con la primera"
                    + " lectura queda encendida la fría y apagada la caliente")
    void encendidasDeAntes_seAjustanConLaPrimeraLectura() throws Exception {
        SWITCHES.fijar("sw-living", "ON");
        SWITCHES.fijar("sw-cocina", "ON");

        cargarSitio(
                POTENCIA_SITIO, habitacion("living", 21.0, 1.0), habitacion("cocina", 19.0, 1.0));
        iniciar();
        temperatura("living", 17.0);
        temperatura("cocina", 23.0);

        esperarSwitch("living", "ON");
        esperarSwitch("cocina", "OFF");
        assertEquals("CALEFACCIONANDO", motivo("living"));
        assertEquals("SATISFECHA", motivo("cocina"));
    }

    @Test
    @DisplayName("Las lecturas anteriores a INICIAR no cuentan: arranca SIN_LECTURA")
    void lecturasAnterioresAIniciar_noCuentan() throws Exception {
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        temperatura("living", 15.0);
        Thread.sleep(300); // que la lectura llegue mientras está detenido

        iniciar();

```

Test 3. Situación inicial: la calefacción ya estaba encendida y todavía no llegó ninguna lectura.

Ambos switches arrancan en ON, como si alguien los hubiera dejado prendidos.

Carga el sitio y arranca el control.

El engine los apaga: sin lectura no se calefacciona a ciegas.

El motivo es SIN_LECTURA.

Fin del test 3.

Test 4. Calefacción encendida de antes, con una habitación fría y otra caliente.

Ambos switches arrancan en ON.

Arranca el control y llegan las primeras lecturas: living a 17 (fría) y cocina a 23 (caliente).

Resultado: la fría queda encendida y la caliente se apaga.

Con sus motivos: CALEFACCIONANDO y SATISFECHA.

Fin del test 4.

Test 5. Las lecturas que llegaron antes de INICIAR no cuentan.

Carga el sitio y publica una lectura con el control detenido.

Espera 300 ms para asegurar que la lectura llegue mientras el control está detenido (la publicación es asincrónica).

Recién ahora se inicia el control.

```
        esperarMotivo("living", "SIN_LECTURA");
        assertSwitch("living", "OFF");
    }

    @Test
    @DisplayName("INICIAR sin sitio configurado no falla; el sitio que llega después se controla")
    void iniciarSinSitio() throws Exception {
        iniciar();
        seMantiene(
                Duration.ofMillis(300),
                () -> SWITCHES.comandos().isEmpty(),
                () -> "comandó sin sitio: " + SWITCHES.comandos());

        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        temperatura("living", 16.0);
        esperarSwitch("living", "ON");
    }

    @Test
    @DisplayName("INICIAR en plena franja punta: no enciende nada, y apaga lo que estaba encendido")
    void iniciarEnFranjaPunta() throws Exception {
        SWITCHES.fijar("sw-living", "ON");
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar("2026-10-01T20:00:00Z", 1);
        temperatura("living", 15.0);

        esperarMotivo("living", "TARIFA_PUNTA");
        esperarSwitch("living", "OFF");
    }

    // ---------------- parada ----------------

    @Test
    @DisplayName(
            "DETENER: deja de comandar; lo que estaba encendido queda como está (§5.2 no exige"
                    + " apagar)")
    void detenerDejaDeComandar() throws Exception {
        cargarSitio(
                POTENCIA_SITIO, habitacion("living", 21.0, 1.0), habitacion("cocina", 19.0, 1.0));
        iniciar();
        temperatura("living", 16.0);
        temperatura("cocina", 15.0);
        esperarSwitch("living", "ON");
        esperarSwitch("cocina", "ON");

        detener();
```

La habitación arranca SIN_LECTURA y apagada: el engine descartó la lectura vieja.

Fin del test 5.

Test 6. INICIAR sin sitio configurado no falla.

Inicia sin inventario y verifica que no se mande ningún comando durante 300 ms.

El sitio llega después, con el control ya en marcha, y se controla normalmente (la habitación fría se enciende).

Fin del test 6.

Test 7. INICIAR en plena franja punta.

El switch estaba encendido; se inicia a las 20:00 de un jueves y llega una lectura fría.

Motivo TARIFA_PUNTA y el switch termina apagado: en punta no solo no se enciende, también se apaga lo que estaba encendido.

Fin del test 7.

Segundo grupo: parada.

Test 8. DETENER deja de comandar, y lo que estaba encendido queda como está (el §5.2 no exige apagar).

Dos habitaciones frías; ambas se encienden.

DETENER.

```
        int comandosAlDetener = SWITCHES.comandos().size();
```
Cuenta los comandos que había hasta ese momento.

```
        temperatura("living", 25.0); // haría apagar, pero ya está detenido
```
Llega una lectura de 25 °C que, con el control en marcha, haría apagar el living.

```

        seMantiene(
                Duration.ofMillis(1500), // más que la reconciliación (1 s)
                () -> SWITCHES.comandos().size() == comandosAlDetener,
                () -> "comandó detenido: " + SWITCHES.comandos());
```
Durante 1.5 s (más que el período de reconciliación de 1 s) no sale ningún comando nuevo.

```
        assertSwitch("living", "ON");
        assertSwitch("cocina", "ON");
```
Los dos switches siguen encendidos: el engine no apaga al detenerse.

```
    }
```
Fin del test 8.

```

    @Test
    @DisplayName("DETENER es idempotente y sin sitio no falla")
    void detenerIdempotente() throws Exception {
```
Test 9. DETENER es idempotente y no falla sin sitio.

```
        detener();
        iniciar();
        detener();
        detener();
```
DETENER sin haber iniciado, INICIAR y DETENER dos veces seguidas; ninguno debe dar error (el helper verifica el 200).

```
        assertTrue(SWITCHES.comandos().isEmpty());
```
No se mandó ningún comando (no hay habitaciones).

```
    }
```
Fin del test 9.

```

    @Test
    @DisplayName(
            "Reiniciar tras DETENER retoma el control: corrige lo que cambió mientras estaba"
                    + " detenido")
    void reiniciarRetomaElControl() throws Exception {
```
Test 10. Al reiniciar después de DETENER se retoma el control y se corrige lo que cambió mientras estaba detenido.

```
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar();
        temperatura("living", 16.0);
        esperarSwitch("living", "ON");
```
Control en marcha y living encendido.

```

        detener();
        SWITCHES.fijar("sw-living", "OFF"); // alguien lo apagó a mano
```
Se detiene y alguien apaga el switch a mano.

```

        iniciar();
        temperatura("living", 16.0);
        esperarSwitch("living", "ON");
```
Al volver a INICIAR, con la lectura fría, el engine lo enciende de nuevo.

```
    }
```
Fin del test 10.

```

    @Test
    @DisplayName(
            "Con el control en marcha, un cambio manual en un switch se corrige al reconciliar")
    void cambioManualSeCorrige() throws Exception {
```
Test 11. Con el control en marcha, un cambio manual en un switch se corrige al reconciliar.

```
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar();
        temperatura("living", 16.0);
        esperarSwitch("living", "ON");

        SWITCHES.fijar("sw-living", "OFF");
        esperarSwitch("living", "ON"); // reconciliación cada 1 s
    }
}
```

Living encendido por el control.

Alguien lo apaga a mano. Sin ninguna lectura nueva, la reconciliación periódica (cada 1 s en el test) detecta la diferencia y lo vuelve a encender.

Fin del test y de la clase.

9. Tests: cambios de configuración en ejecución

modules/engine/src/test/java/uy/edu/utec/iiss/engine/integracion/CambioConfiguracionTest.java — líneas 1–174

```java
package uy.edu.utec.iiss.engine.integracion;

import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import java.time.Duration;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import uy.edu.utec.iiss.engine.inventario.Inventario;

```

Paquete e imports, incluidos los builders de MockMvc para POST y DELETE.

```java
/**
 * Cambios de configuración con el control en ejecución (§5.1 en caliente): se adoptan sin reiniciar
 * y sin perder las lecturas que ya había.
 */
```

Javadoc: los cambios de configuración en caliente (§5.1) se adoptan sin reiniciar el control y sin perder las lecturas que ya había.

```java
@DisplayName("Integración: cambios de configuración en ejecución")
class CambioConfiguracionTest extends IntegracionBase {

```

Nombre en el reporte y herencia de la base.

```java
    @Test
    @DisplayName("Bajar la temperatura esperada por debajo de la actual apaga la habitación")
    void bajarTemperaturaEsperada_apaga() throws Exception {
```

Test 1. Bajar la temperatura esperada por debajo de la actual apaga la habitación.

```java
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar();
        temperatura("living", 19.0);
        esperarSwitch("living", "ON");

```

Living con esperada 21 y lectura 19: se enciende.

```java
        modificarHabitacion(habitacion("living", 18.0, 1.0));
```

PUT /habitaciones/living con la esperada en 18.

```java
        esperarSwitch("living", "OFF");
        esperarMotivo("living", "SATISFECHA");
```

Se apaga con motivo SATISFECHA, sin necesidad de una lectura nueva.

```java
    }

```

Fin del test 1.

```java
    @Test
    @DisplayName(
            "Subir la temperatura esperada enciende la habitación con la lectura que ya tenía, sin"
                    + " esperar una nueva")
    void subirTemperaturaEsperada_enciende() throws Exception {
```

Test 2. Subir la temperatura esperada enciende la habitación con la lectura que ya tenía.

```java
        cargarSitio(POTENCIA_SITIO, habitacion("living", 18.0, 1.0));
        iniciar();
        temperatura("living", 19.0);
        esperarMotivo("living", "SATISFECHA");

```

Esperada 18 y lectura 19: está SATISFECHA.

```
        modificarHabitacion(habitacion("living", 22.0, 1.0));
        esperarSwitch("living", "ON");
```

Se sube la esperada a 22 y la habitación se enciende con la lectura de 19 que ya tenía.

```
    }
```

Fin del test 2.

```

    @Test
    @DisplayName("Una habitación agregada en caliente se suscribe a su termostato y se controla")
    void agregarHabitacion() throws Exception {
```

Test 3. Una habitación agregada en caliente se controla.

```
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar();
        temperatura("living", 16.0);
        esperarSwitch("living", "ON");

```

Control en marcha con living encendido.

```
        mvc.perform(
                        conClave(post("/habitaciones"))
                                .content(json.writeValueAsString(habitacion("cocina", 20.0, 1.0))))
                .andExpect(status().isCreated());
```

POST /habitaciones con la cocina; se espera 201 Created.

```
        temperatura("cocina", 15.0);
        esperarSwitch("cocina", "ON");
```

El engine se suscribió a su termostato (el helper lo espera) y, con una lectura fría, la enciende.

```
        assertSwitch("living", "ON");
```

Living no se vio afectado.

```
    }
```

Fin del test 3.

```

    @Test
    @DisplayName(
            "Una habitación eliminada en caliente deja de controlarse: ni se comanda ni se escucha"
                    + " su termostato")
    void eliminarHabitacion() throws Exception {
```

Test 4. Una habitación eliminada en caliente deja de controlarse: ni se comanda su switch ni se escucha su termostato.

```
        cargarSitio(
                POTENCIA_SITIO, habitacion("living", 21.0, 1.0), habitacion("cocina", 20.0, 1.0));
        iniciar();
        temperatura("living", 16.0);
        temperatura("cocina", 16.0);
        esperarSwitch("cocina", "ON");

```

Dos habitaciones encendidas.

```
        mvc.perform(conClave(delete("/habitaciones/cocina"))).andExpect(status().isNoContent());
```

DELETE /habitaciones/cocina; se espera 204 No Content.

```
        esperar(
                () -> !BROKER.haySuscriptor(topic("cocina")),
                () -> "sigue suscrito al termostato de la cocina");
```

El engine se desuscribe del topic de la cocina (lo informa el broker).

```
        int comandosACocina = SWITCHES.comandosA("sw-cocina").size();
        BROKER.publicarTemperatura(topic("cocina"), 30.0);

```

Cuenta los comandos a la cocina y publica igual una lectura en su topic.

```
        seMantiene(
                Duration.ofMillis(1500), // más que una reconciliación
                () -> SWITCHES.comandosA("sw-cocina").size() == comandosACocina,
                () -> "comandó el switch de una habitación eliminada: " + SWITCHES.comandos());
```

Durante 1.5 s no sale ningún comando hacia el switch de la cocina, ni siquiera en la reconciliación.

```
        assertTrue(adaptador.estados().stream().noneMatch(e -> e.idHabitacion().equals("cocina")));
```

La cocina ya no aparece en el estado del adaptador.

```
    }
```
Fin del test 4.

```

    @Test
    @DisplayName(
            "Bajar la potencia contratada en caliente apaga la de menor déficit y respeta el nuevo"
                    + " límite")
    void bajarPotenciaContratada() throws Exception {
```
Test 5. Bajar la potencia contratada en caliente.

```
        Inventario holgado =
                inventario(
                        POTENCIA_SITIO,
                        habitacion("living", 22.0, 1.5),
                        habitacion("dormitorio", 22.0, 1.5));
        cargarSitio(holgado);
```
Dos habitaciones de 1.5 kW con 3.7 kW contratados: entran las dos.

```
        iniciar();
        temperatura("living", 16.0); // déficit 6
        temperatura("dormitorio", 20.0); // déficit 2
        esperarSwitch("living", "ON");
        esperarSwitch("dormitorio", "ON");

```
Living con déficit 6 y dormitorio con déficit 2; las dos encendidas.

```
        Inventario ajustado =
                inventario(2.0, holgado.habitaciones().toArray(Inventario.Habitacion[]::new));
        cargarSitio(ajustado);
```
Se vuelve a cargar el mismo sitio, pero con 2.0 kW contratados (entra una sola).

```
        esperarSwitch("dormitorio", "OFF");
        esperarMotivo("dormitorio", "SIN_POTENCIA_DISPONIBLE");
```
Se apaga el dormitorio, el de menor déficit, con motivo SIN_POTENCIA_DISPONIBLE.

```
        assertSwitch("living", "ON");
```
Living, el de mayor déficit, sigue encendido.

```
    }

    @Test
    @DisplayName(
            "Cambiar el switch de una habitación: el nuevo se comanda y el viejo no se toca más")
    void cambiarSwitch() throws Exception {
```
Test 6. Cambiar el switch de una habitación: el nuevo se comanda y el viejo no se toca más.

```
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar();
        temperatura("living", 16.0);
        esperarSwitch("living", "ON");

```
Living encendido con su switch original "sw-living".

```
        Inventario.Habitacion h = habitacion("living", 21.0, 1.0);
        modificarHabitacion(
                new Inventario.Habitacion(
                        h.id(),
                        h.nombre(),
                        h.temperaturaEsperada(),
                        h.potenciaKW(),
                        h.idTermostato(),
                        h.topicTermostato(),
                        "sw-living-nuevo",
                        h.urlSwitch()));
```
PUT de la habitación igual, pero con el switch "sw-living-nuevo".

```
        esperar(
                () -> SWITCHES.estado("sw-living-nuevo").equals("ON"),
                () -> "no se encendió el switch nuevo: " + SWITCHES.comandos());
```

El switch nuevo recibe el ON (el engine olvidó lo que sabía del viejo y comanda el nuevo).

```
        int comandosAlViejo = SWITCHES.comandosA("sw-living").size();
```

Cuenta los comandos que tenía el switch viejo.

```
        temperatura("living", 25.0);
        esperar(
                () -> SWITCHES.estado("sw-living-nuevo").equals("OFF"),
                () -> "no se apagó el switch nuevo: " + SWITCHES.comandos());
```

Una lectura caliente apaga el switch nuevo.

```
        assertTrue(
                SWITCHES.comandosA("sw-living").size() == comandosAlViejo,
                "se siguió comandando el switch viejo: " + SWITCHES.comandos());
```

El viejo no recibió ningún comando más.

```
    }
```

Fin del test 6.

```

```

```
    @Test
    @DisplayName("Cambiar el topic del termostato: se escucha el nuevo y se deja el viejo")
    void cambiarTopic() throws Exception {
```

Test 7. Cambiar el topic del termostato.

```
        cargarSitio(POTENCIA_SITIO, habitacion("living", 21.0, 1.0));
        iniciar();
        temperatura("living", 16.0);
        esperarSwitch("living", "ON");
```

Living encendido, escuchando su topic original.

```

```

```
        Inventario.Habitacion h = habitacion("living", 21.0, 1.0);
        String topicNuevo = "ht-living-v2/status/temperature:0";
        modificarHabitacion(
                new Inventario.Habitacion(
                        h.id(),
                        h.nombre(),
                        h.temperaturaEsperada(),
                        h.potenciaKW(),
                        "ht-living-v2",
                        topicNuevo,
                        h.idSwitch(),
                        h.urlSwitch()));
```

PUT de la habitación con termostato "ht-living-v2" y su topic nuevo.

```
        esperar(() -> BROKER.haySuscriptor(topicNuevo), () -> "no se suscribió al topic nuevo");
        esperar(() -> !BROKER.haySuscriptor(topic("living")), () -> "sigue en el topic viejo");
```

El engine se suscribe al topic nuevo y se desuscribe del viejo.

```

```

```
        BROKER.publicarTemperatura(topicNuevo, 25.0);
        esperarSwitch("living", "OFF");
```

Una lectura caliente por el topic nuevo apaga la habitación: la lectura se tomó del topic correcto.

```
    }
}
```

Fin del test y de la clase.